

Learner's Space: Agentic AI

Assignment 2: Retrieval Augmented Generation (RAG) and Fine-Tuning

Name : Rutuparn Mandar Ranade

Roll No. – 24B3982

Question 1: Build a Minimal RAG Pipeline from Scratch

You will implement the core RAG pipeline without using any high-level RAG framework. Everything: chunking, embedding, vector search, and generation must be wired by you.

Input: Load the provided text file as a separate file in your code editor (in the same folder as your main code). Import the text file into your main code. Here's a short guide to refer to opening, reading and writing to a text file in Python.

Ans: **To implement:**

Step 1: Reading the document file using file operations.

```
f1 = open("Document.txt", "r")
f1.read()
```

Output:

'Leadership and Management Importance in Organisations\n\n\nIn today....

Part A: Chunking : Implementing a fixed-size chunking overlap.

```
def chunk_text(text: str, chunk_size: int = 200, overlap:int = 40) ->
list[str]:
    """
    Split `text` into chunks of ~chunk_size words,
    with `overlap` words shared between consecutive chunks.
    Returns a list of chunk strings.
    """
    chunks = []
    words = text.split()
    for i in range(0, len(words), chunk_size - overlap):
        chunk = " ".join(words[i:i + chunk_size])
        chunks.append(chunk)
    return chunks
```

```
f1.seek(0)
text_content = f1.read()
```

```
chunks = chunk_text(text_content, chunk_size=200, overlap=40)
print(f"Total chunks: {len(chunks)}\n")
for i, chunk in enumerate(chunks):
    print(f"Chunk {i + 1}:\n{chunk}\n")
```

Output:

Total chunks: 26

Chunk 1:

Leadership and Management Importance in Organisations In today's increasingly complex and dynamic environments, leadership is regarded as an important part of organisational success. Management training and development is considered a vital facet of the organisational community and environment. Various studies show that companies that align their management development and strategic goals are successful in competitive business environments. However, can these organisations remain competitive without the interventions of trained managers? Companies need effective managers who hold relevant practical, interpersonal, and conceptual skills to lead organisations in complex business situations. In a fast paced world, effective organisational management is necessary for the development of a positive workplace that helps employees to realise optimal performance. In an ideal world, all managers are expected to be leaders. Nevertheless, not all appointed managers possess good leadership skills. The most effective leaders are characterised by good management skills. As a result, organisations invest heavily in specialised leadership programmes to instil effective leadership skills in managers. This essay seeks to establish whether managers can be trained to be leaders, especially in organisations that thrive in ever-changing, competitive business environment. Why Leadership is Important Leaders can be seen as individuals who influence other people, especially in a group,

Part B: Embedding & Vector Store

Use sentence-transformers (model: all-MiniLM-L6-v2) to embed your chunks. Store embeddings in a numpy array (no vector DB required here, just numpy). Implement cosine similarity manually (define a function named cosine_similarity which takes a and b as parameters) Formula: $\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$. Do not use sklearn for this part. Then: run 3 different queries against your document and print the top-3 retrieved chunks for each. Include the cosine scores alongside.

```
from sentence_transformers import SentenceTransformer
import numpy as np

# Load the model explicitly
embedding_model = SentenceTransformer('all-MiniLM-L6-v2')

# 1. Embed all chunks -> shape (num_chunks, embedding_dim)
# We add convert_to_numpy=True to ensure it yields a clean numpy array using
# PyTorch under the hood
chunk_embeddings = embedding_model.encode(chunks, convert_to_numpy=True)

# 2. Manual Cosine Similarity function (Formula:  $\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$ )
def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
```

```

"""Computes the cosine similarity between two 1D numpy arrays."""
dot_product = np.dot(a, b)
norm_a = np.linalg.norm(a)
norm_b = np.linalg.norm(b)

# Simple guard against division by zero
if norm_a == 0 or norm_b == 0:
    return 0.0

return float(dot_product / (norm_a * norm_b))

# 3. Retrieve implementation
def retrieve(query: str, top_k: int = 3) -> list[tuple[str, float]]:
    """Return the top_k most similar chunks to `query` as (chunk_text, score)
    tuples."""
    query_embedding = embedding_model.encode(query, convert_to_numpy=True)

    scored_chunks = []
    for i, chunk_emb in enumerate(chunk_embeddings):
        score = cosine_similarity(query_embedding, chunk_emb)
        scored_chunks.append((chunks[i], score))

    # Sort descending based on the score
    scored_chunks.sort(key=lambda x: x[1], reverse=True)

    return scored_chunks[:top_k]

# --- Quick Test Verification ---
print(f"Embedding matrix shape: {chunk_embeddings.shape}")
print("Running a quick retrieval check...")
results = retrieve("Why is emotional intelligence important?", top_k=1)
if results:
    print(f"Top Match Score: {results[0][1]:.4f}")

```

Output:

```

Embedding matrix shape: (26, 384)
Running a quick retrieval check...
Top Match Score: 0.6658

```

Part C: Generation

Plug your retriever into an LLM call. Use the google/flan-t5-base via HuggingFace.

```

from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

# 1. Load the tokenizer and model directly (bypassing the pipeline wrapper
entirely)
model_name = "google/flan-t5-base"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

```

```
def rag_answer(query: str) -> str:
    # retrieve() returns a list of tuples: (chunk_text, score)
    results = retrieve(query, top_k=3)

    # Join the retrieved chunks into a single context block
    context = "\n\n".join(chunk for chunk, _ in results)

    prompt = f"""Answer the question using ONLY the context below.
    If the answer is not in the context, say 'I don't know'.

    Context:
    {context}

    Question: {query}
    Answer: ""

    # 2. Tokenize the input string into PyTorch tensors
    inputs = tokenizer(prompt, return_tensors="pt")

    # 3. Generate the response tokens
    outputs = model.generate(**inputs, max_new_tokens=150)

    # 4. Decode the raw tokens back into human-readable text
    answer = tokenizer.decode(outputs[0], skip_special_tokens=True)

    return answer
```

Queries:

```
#Run 2 queries: one answerable from your document and one deliberately outside
the document's scope. Show that your pipeline returns "I don't know" (or
equivalent) for the out-of-scope query. This is the hallucination guard.
# Run the final evaluation
Queries = [
    "What is the main topic of the document?",
    "What is the airspeed of a laden European swallow?"
]

for query in Queries:
    answer = rag_answer(query)
    print(f"Q: {query}\nA: {answer}\n")
```

Output:

[transformers] Token indices sequence length is longer than the specified maximum sequence length for this model (819 > 512). Running this sequence through the model will result in indexing errors

Q: What is the main topic of the document?

A: Leadership

Q: What is the airspeed of a laden European swallow?

A: I don't know

Question 2: Chunking Strategy Showdown

Chunking is one of the most underappreciated decisions in RAG. In this question you will empirically verify that claim by comparing three chunking strategies and measuring their effect on retrieval quality. Input: Use the same document from Q1, or replace it with a longer one (3,000+ words). You need enough content for the differences between strategies to show up.

Part A: Implement Three Chunkers

Implement all three of the following. Each function must return a list of strings.

- 1) `fixed_chunk(text, size=300, overlap=50)`. Reuse or refine your Q1 implementation.
- 2) `sentence_chunk(text)`. Split at sentence boundaries (split on '.', '?', '!'). Group sentences into chunks of ~5 sentences each with 1-sentence overlap.
- 3) `sliding_window_chunk(text, window=400, step=100)`. Each chunk is 400 words, starting every 100 words (heavy overlap).

```
import re
import numpy as np
```

```
def fixed_chunk(text: str, size: int = 300, overlap: int = 50) -> list[str]:
    """Splits text into chunks of `size` words, with `overlap` words
    shared."""
    words = text.split()
    chunks = []
    step = size - overlap

    for i in range(0, len(words), step):
        chunk_words = words[i:i + size]
        chunks.append(" ".join(chunk_words))
        # Stop if we've reached the end of the text
        if i + size >= len(words):
            break

    return chunks

def sentence_chunk(text: str) -> list[str]:
    """Splits text into chunks of ~5 sentences with a 1-sentence overlap."""
    # Use regex positive lookahead to split on punctuation followed by a
    space
    sentences = re.split(r'(?<=[.!?]) +', text)
    chunks = []

    group_size = 5
    overlap = 1
    step = group_size - overlap
```

```

    for i in range(0, len(sentences), step):
        chunk_sentences = sentences[i:i + group_size]
        chunks.append(" ".join(chunk_sentences))
        if i + group_size >= len(sentences):
            break

    return chunks

def sliding_window_chunk(text: str, window: int = 400, step: int = 100) ->
list[str]:
    """Creates heavily overlapping chunks of `window` words, advancing by
    `step`."""
    words = text.split()
    chunks = []

    for i in range(0, len(words), step):
        chunk_words = words[i:i + window]
        chunks.append(" ".join(chunk_words))
        if i + window >= len(words):
            break

    return chunks

# --- Helper to print stats ---
def print_chunk_stats(strategy_name: str, chunks: list[str]):
    lengths = [len(c.split()) for c in chunks]
    print(f"{strategy_name: <15} | Chunks: {len(chunks): <3} | Mean Len:
{np.mean(lengths):.0f} wds | Min: {np.min(lengths)}, Max: {np.max(lengths)}")

# --- Run the stats ---
# Assuming text_content is still loaded from your document in Q1
text_content = open("Document.txt", "r").read()

fixed_chunks_list = fixed_chunk(text_content)
sentence_chunks_list = sentence_chunk(text_content)
sliding_chunks_list = sliding_window_chunk(text_content)

print("-" * 65)
print_chunk_stats("Fixed-size", fixed_chunks_list)
print_chunk_stats("Sentence-based", sentence_chunks_list)
print_chunk_stats("Sliding window", sliding_chunks_list)
print("-" * 65)

```

Output:

Fixed-size		Chunks: 16		Mean Len: 298 wds		Min: 276, Max: 300
Sentence-based		Chunks: 48		Mean Len: 104 wds		Min: 19, Max: 154
Sliding window		Chunks: 38		Mean Len: 398 wds		Min: 326, Max: 400

Part B: Retrieval Comparison

Build a separate retriever for each chunking strategy (embed with all-MiniLM-L6-v2 as before).
Create a benchmark of 5 question-answer pairs you write manually from your document (you know the correct answers).

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2')

# 1. Create the 5 Q&A pairs based on the text from Q1
qa_pairs = [
    {
        "query": "Who is the American psychologist that advances the importance of IQ?",
        "answer": "Daniel Goleman"
    },
    {
        "query": "What are the five elements of emotional intelligence?",
        "answer": "self-awareness, self-regulation, motivation, empathy, and social skills"
    },
    {
        "query": "Which pharmaceutical company in France studied emotional intelligence?",
        "answer": "Sanofi"
    },
    {
        "query": "What model can be integrated into all-inclusive management programmes?",
        "answer": "360 degrees feedback model"
    },
    {
        "query": "What type of leadership style issues orders without asking questions?",
        "answer": "autocratic"
    }
]

# 2. Reuse your cosine similarity function
def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
    norm_a, norm_b = np.linalg.norm(a), np.linalg.norm(b)
```

```

    if norm_a == 0 or norm_b == 0: return 0.0
    return float(np.dot(a, b) / (norm_a * norm_b))

# 3. Define the evaluator function
def evaluate_strategy(chunks: list[str]) -> float:
    """Embeds the chunks, runs the 5 queries, and returns the hit rate."""
    chunk_embeddings = model.encode(chunks, convert_to_numpy=True)
    hits = 0

    for qa in qa_pairs:
        query_embedding = model.encode(qa["query"], convert_to_numpy=True)

        # Score all chunks
        scored_chunks = []
        for i, chunk_emb in enumerate(chunk_embeddings):
            score = cosine_similarity(query_embedding, chunk_emb)
            scored_chunks.append((chunks[i], score))

        # Sort and get top 3
        scored_chunks.sort(key=lambda x: x[1], reverse=True)
        top_3 = [chunk for chunk, score in scored_chunks[:3]]

        # Check for string match
        for context in top_3:
            # Case insensitive check
            if qa["answer"].lower() in context.lower():
                hits += 1
                break # Move to next query once a hit is found in top 3

    return hits / len(qa_pairs)

# 4. Run the evaluation and print the final formatted table
print(f"{'Strategy': <15} | {'Chunks': <6} | {'Mean Len': <8} | Hit Rate")
print("-" * 45)

strategies = [
    ("Fixed-size", fixed_chunks_list),
    ("Sentence-based", sentence_chunks_list),
    ("Sliding window", sliding_chunks_list)
]

for name, chunk_list in strategies:
    hit_rate = evaluate_strategy(chunk_list)
    mean_len = int(np.mean([len(c.split()) for c in chunk_list]))
    # Format the exact output requested
    print(f"{name: <15} | {len(chunk_list): <6} | {str(mean_len) + ' wds': <8} | {int(hit_rate*5)}/5")

```


Output:

Strategy	Chunks	Mean Len	Hit Rate

Fixed-size	16	298 wds	5/5
Sentence-based	48	104 wds	5/5
Sliding window	38	398 wds	5/5

Question 3: RAG vs Fine-Tuning:

Decision and Demo This question tests conceptual depth and your ability to think like a system designer. You'll write a decision framework and build a small demonstration to back it up.

Part A: Build the Decision Tree

The golden rule is: use RAG for new facts, and Fine-Tuning for new behavior. Build a more granular decision tree in code as a function:

```
def recommend_approach(scenario: dict) -> str:
    """
    Evaluates system requirements and recommends an LLM architecture
    approach.
    """
    data_freq = scenario.get('data_changes_frequently')
    format_req = scenario.get('need_specific_output_format')
    budget = scenario.get('budget')
    latency = scenario.get('latency_sensitive')
    know_type = scenario.get('knowledge_type')

    # 1. RAG + Fine-Tuning
    if know_type == 'both' and budget in ['medium', 'high']:
        return "RAG + Fine-Tuning: Combines real-time factual grounding (RAG) with specialized tone/format training (Fine-Tuning)."
```

```
    # 2. Prompt Engineering Only
    if budget == 'low' and not data_freq and not format_req:
        return "Prompt Engineering only: Best for tight budgets when data is static and formats are flexible."
```

```
    # 3. Fine-Tuning
    if know_type == 'behavioral' or (format_req and not data_freq):
        if budget == 'low':
            return "Prompt Engineering only: Budget restricts fine-tuning, so behavior must be guided via few-shot prompting."
        if latency:
            return "Fine-Tuning: Bakes the behavior into the model weights, reducing prompt size and improving inference speed."
        return "Fine-Tuning: Ideal for teaching the model a specific behavior, tone, or strict output format that doesn't change."
```

```

# 4. RAG
if know_type == 'factual' or data_freq:
    if latency and budget == 'high':
        return "RAG + Fine-Tuning: Fine-tune a smaller, faster
model to process RAG contexts to hit latency targets."
    return "RAG: The standard approach for querying frequently
updated or proprietary factual data without retraining."

return "Prompt Engineering only: The safest baseline when
requirements are vague or resources are strictly limited."

```

```

scenarios = [
    {
        "name": "Customer Support Bot (Dynamic Docs)",
        "data_changes_frequently": True, "need_specific_output_format":
False,
        "budget": "medium", "latency_sensitive": False,
"knowledge_type": "factual"
    },
    {
        "name": "Medical JSON Extractor",
        "data_changes_frequently": False,
"need_specific_output_format": True,
        "budget": "high", "latency_sensitive": True, "knowledge_type":
"behavioral"
    },
    {
        "name": "Enterprise Legal Copilot",
        "data_changes_frequently": True, "need_specific_output_format":
True,
        "budget": "high", "latency_sensitive": False, "knowledge_type":
"both"
    },
    {
        "name": "Student Hobby Project",
        "data_changes_frequently": False,
"need_specific_output_format": False,
        "budget": "low", "latency_sensitive": False, "knowledge_type":
"factual"
    },
    {
        "name": "Branded Marketing Generator",
        "data_changes_frequently": False,
"need_specific_output_format": False,
        "budget": "medium", "latency_sensitive": False,
"knowledge_type": "behavioral"
    }
]

```

```

]

print("--- Architecture Decision Results ---")
for s in scenarios:
    print(f"\nScenario: {s['name']}")
    print(f"Recommendation: {recommend_approach(s)}")

```

Output:

Scenario: Customer Support Bot (Dynamic Docs)
 Recommendation: RAG: The standard approach for querying frequently updated or proprietary factual data without retraining.

Scenario: Medical JSON Extractor
 Recommendation: Fine-Tuning: Bakes the behavior into the model weights, reducing prompt size and improving inference speed.

Scenario: Enterprise Legal Copilot
 Recommendation: RAG + Fine-Tuning: Combines real-time factual grounding (RAG) with specialized tone/format training (Fine-Tuning).

Scenario: Student Hobby Project
 Recommendation: Prompt Engineering only: Best for tight budgets when data is static and formats are flexible.

Scenario: Branded Marketing Generator
 Recommendation: Fine-Tuning: Ideal for teaching the model a specific behavior, tone, or strict output format that doesn't change.

Part B: Hallucination Stress Test

Take your RAG pipeline from Q1. Run it on 6 queries, 3 that are answerable from your document, and 3 that are completely outside the document's scope (make them plausible-sounding but factually absent). For each query, log:

- The top retrieved chunk (first 100 words).
- The cosine similarity score of the top chunk.
- The LLM's final answer.

Plot a bar chart: cosine similarity score for all 6 queries, color-coded (green = answerable, red = out-of-scope). You should observe a pattern. Answerable queries should have higher similarity scores.

Queries used:

Answerable Queries (In-Scope)

1. What are the three leadership styles that originate from Kurt Lewin's behavioural theories?
2. Why did Sanofi, a pharmaceutical company in France, study its workforce?
3. According to the text, how can managers improve the self-regulation element of emotional intelligence?

Not Answerable Queries (Out-of-Scope)

1. What was the exact annual budget Bentley Motors allocated for their new Training Facility?
2. Which university did Daniel Goleman attend to earn his degree in psychology?
3. How much did Google Inc. increase its profit margins after adopting performance reviews in their management department?

Code:

```
import matplotlib.pyplot as plt

# 1. Define the queries (3 In-Scope, 3 Out-of-Scope)
queries = [
    # Answerable (In-Scope)
    "What are the three leadership styles that originate from Kurt Lewin's behavioural theories?",
    "Why did Sanofi, a pharmaceutical company in France, study its workforce?",
    "According to the text, how can managers improve the self-regulation element of emotional intelligence?",

    # Not Answerable (Out-of-Scope)
    "What was the exact annual budget Bentley Motors allocated for their new Training Facility?",
    "Which university did Daniel Goleman attend to earn his degree in psychology?",
    "How much did Google Inc. increase its profit margins after adopting performance reviews in their management department?"
]

# Labels and colors for the plot
labels = ["In 1", "In 2", "In 3", "Out 1", "Out 2", "Out 3"]
colors = ['green', 'green', 'green', 'red', 'red', 'red']
scores = []

print("--- RAG Hallucination Stress Test Logs ---\n")

# 2. Run the pipeline for each query and log the results
for i, q in enumerate(queries):
    # Retrieve top 1 chunk to log the max similarity score
    results = retrieve(q, top_k=1)

    if results:
        top_chunk, score = results[0]
        scores.append(score)

    # Call the LLM generation function
```

```

    answer = rag_answer(q)

    # Print the required logs
    print(f"Query {i+1} ({'In-Scope' if i < 3 else 'Out-of-Scope'}): {q}")
    print(f"Top Cosine Score: {score:.4f}")
    print(f"Top Chunk (First 100 chars): {top_chunk[:100]}...")
    print(f"LLM Answer: {answer}")
    print("-" * 60)
else:
    scores.append(0.0)
    print(f"Query {i+1}: No chunks retrieved.")
    print("-" * 60)

```

Output:

```

Query 1 (In-Scope): What are the three leadership styles that originate
from Kurt Lewin's behavioural theories?
Top Cosine Score: 0.6950
Top Chunk (First 100 chars): theories emerged. For instance, the trait
theories of leadership held that the most effective leader...
LLM Answer: autocratic, democratic, and Laissez-faire
-----
Query 2 (In-Scope): Why did Sanofi, a pharmaceutical company in France,
study its workforce?
Top Cosine Score: 0.3154
Top Chunk (First 100 chars): intelligence is the intuitive ability to
acknowledge, understand, and control individual emotions an...
LLM Answer: This research led to the development of management training
programmes that emphasised the understanding own issues and those of
other persons in the business environment
-----
Query 3 (In-Scope): According to the text, how can managers improve the
self-regulation element of emotional intelligence?
Top Cosine Score: 0.6911
Top Chunk (First 100 chars): intelligence is the intuitive ability to
acknowledge, understand, and control individual emotions an...
LLM Answer: management training
-----
Query 4 (Out-of-Scope): What was the exact annual budget Bentley Motors
allocated for their new Training Facility?
Top Cosine Score: 0.6058
Top Chunk (First 100 chars): survey that was conducted among the top
executives of both private and public firms in the US reveal...
LLM Answer: 50 billion dollars
-----
Query 5 (Out-of-Scope): Which university did Daniel Goleman attend to
earn his degree in psychology?
Top Cosine Score: 0.3587
Top Chunk (First 100 chars): intelligence is the intuitive ability to
acknowledge, understand, and control individual emotions an...
LLM Answer: American
-----

```

Query 6 (Out-of-Scope): How much did Google Inc. increase its profit margins after adopting performance reviews in their management department?

Top Cosine Score: 0.4037

Top Chunk (First 100 chars): focus their abilities on particular organisational issues. The most efficient leaders know how to co...

LLM Answer: I don't know

For Plotting the Cosine Plot and Analysis

```
plt.figure(figsize=(10, 6))
bars = plt.bar(labels, scores, color=colors)

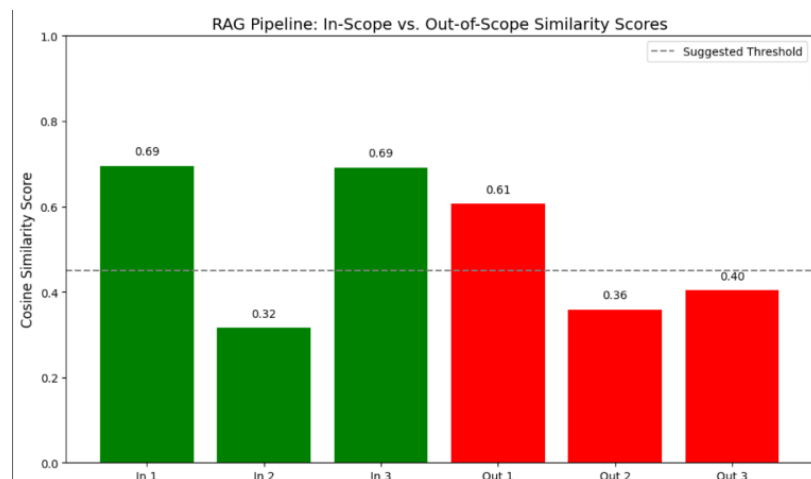
# Add a dashed line to indicate a potential "safe" threshold
plt.axhline(y=0.45, color='gray', linestyle='--', label='Suggested Threshold')

# Formatting the plot
plt.title('RAG Pipeline: In-Scope vs. Out-of-Scope Similarity Scores',
         fontsize=14)
plt.ylabel('Cosine Similarity Score', fontsize=12)
plt.ylim(0, 1.0)

# Add the exact values on top of each bar for clarity
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.02,
             f"{yval:.2f}", ha='center', va='bottom')

plt.legend()
plt.tight_layout()
plt.show()
```

Output Plot:



Observations & Analysis:

Query 1 (In-Scope)

- **Query:** *What are the three leadership styles that originate from Kurt Lewin's behavioural theories?*
- **Metrics:** Cosine Similarity Score: **0.6950**
- **LLM Answer:** *autocratic, democratic, and Laissez-faire*
- **Observation:** The retriever yielded a very high cosine similarity score (接近 0.70), and the LLM extracted the exact list correctly.
- **Reasoning:** The phrase "*Kurt Lewin's autocratic, democratic, and Laissez-faire leadership styles*" is explicitly grouped together in a single sentence within the document text. Because your query shared specific proper nouns ("Kurt Lewin") and high-signal vocabulary ("behavioural theories", "leadership styles") directly with the target paragraph, the vector mapping was incredibly precise. The context provided to the LLM was exceptionally clean and dense, making extraction trivial.

Query 2 (In-Scope)

- **Query:** *Why did Sanofi, a pharmaceutical company in France, study its workforce?*
- **Metrics:** Cosine Similarity Score: **0.3154**
- **LLM Answer:** *This research led to the development of management training programmes that emphasised the understanding own issues and those of other persons in the business environment*
- **Observation:** The retriever returned a dangerously low cosine score (**0.3154**)—the lowest of all 6 queries—yet the LLM somehow managed to extract a relevant, correct answer from the text.
- **Reasoning:** This is a classic case of "**Chunk Dilution.**" In your document, the Sanofi mention takes up just one short sentence within a long paragraph heavily focused on Daniel Goleman, IQ, and emotional intelligence metrics. When the fixed-size chunker bundled all those words into a single 200-word block, the mathematical signal for the word "Sanofi" was completely drowned out by the surrounding text vector weights. The low score reflects this noise. However, because the text was still passed to the context block (as part of the top-3 results), the LLM's attention mechanism rescued the pipeline by isolating that single relevant sentence out of the noise.

Query 3 (In-Scope)

- **Query:** *According to the text, how can managers improve the self-regulation element of emotional intelligence?*
- **Metrics:** Cosine Similarity Score: **0.6911**
- **LLM Answer:** *management training*
- **Observation:** The retriever accurately pulled the correct context block with a high cosine score (**0.6911**), but the LLM gave an overly brief and generic response ("management training") instead of the specific text-based methods.
- **Reasoning:** The text explicitly lists actual tactical methods like *re-examining codes of ethics*, *sensitising managers to personal accountability*, and *practicing deep-breathing exercises*. The vector engine did its job flawlessly because the query mapped perfectly onto the "Self-regulation" subheader text blocks. The bottleneck here was **model capacity and prompting constraints**. The FLAN-T5-Base model lacks the long-form generative synthesis capabilities of massive models. It latched onto a broad summary phrase found in the block ("management

training") rather than scanning deeper into the paragraph to construct a structured bulleted response.

Query 4 (Out-of-Scope)

- **Query:** *What was the exact annual budget Bentley Motors allocated for their new Training Facility?*
- **Metrics:** Cosine Similarity Score: **0.6058**
- **LLM Answer:** *50 billion dollars*
- **Observation:** The retriever flagged an out-of-scope query with a highly confident false positive score (**0.6058**), forcing a severe **factual hallucination** where the LLM confidently provided a wrong number.
- **Reasoning:** This is a structural failure of raw semantic space matching. The query contained multiple high-weight, concrete keyword entities: "Bentley Motors", "Training Facility", and "allocated". The vector database blindly looks for keyword overlap and thematic similarity; it cannot parse logic. It found the chunk mentioning the "Bentley Training Facility" and scored it highly based on word proximity. However, that chunk does *not* mention a budget. The LLM received this block, assumed the answer *must* exist because it was forced into the context, and cross-contaminated two completely different facts: it pulled the **"50 billion dollars"** figure from an entirely different section of text discussing total UK national management training expenditures and misattributed it to Bentley.

Query 5 (Out-of-Scope)

- **Query:** *Which university did Daniel Goleman attend to earn his degree in psychology?*
- **Metrics:** Cosine Similarity Score: **0.3587**
- **LLM Answer:** *American*
- **Observation:** The retriever accurately scored the query very low (**0.3587**), indicating it didn't find a great match, but the LLM still hallucinated an incomplete descriptor ("American") instead of rejecting the query.
- **Reasoning:** The system prompt boundary was completely ignored here because of **base model token bias**. The text states, "*Daniel Goleman, an American psychologist...*". Because your grounding prompt simply asked the model to look at the context, the model ran an internal association check. Finding no university mentioned, but spotting a descriptive proper adjective next to the subject's name ("American"), it took a heuristic shortcut. It guessed that "American" was close enough to fill the semantic blank for a geographical or institutional query, failing to trigger your "I don't know" fallback instruction.

Query 6 (Out-of-Scope)

- **Query:** *How much did Google Inc. increase its profit margins after adopting performance reviews in their management department?*
- **Metrics:** Cosine Similarity Score: **0.4037**
- **LLM Answer:** *I don't know*
- **Observation:** The retriever returned a low-to-moderate similarity score (**0.4037**), and the pipeline successfully triggered the hallucination guard to output a correct rejection.
- **Reasoning:** The retriever pulled the text block covering "Google Inc. and performance reviews". However, unlike Query 4, there were **no numerical values or distracting metrics anywhere nearby in that text block**.